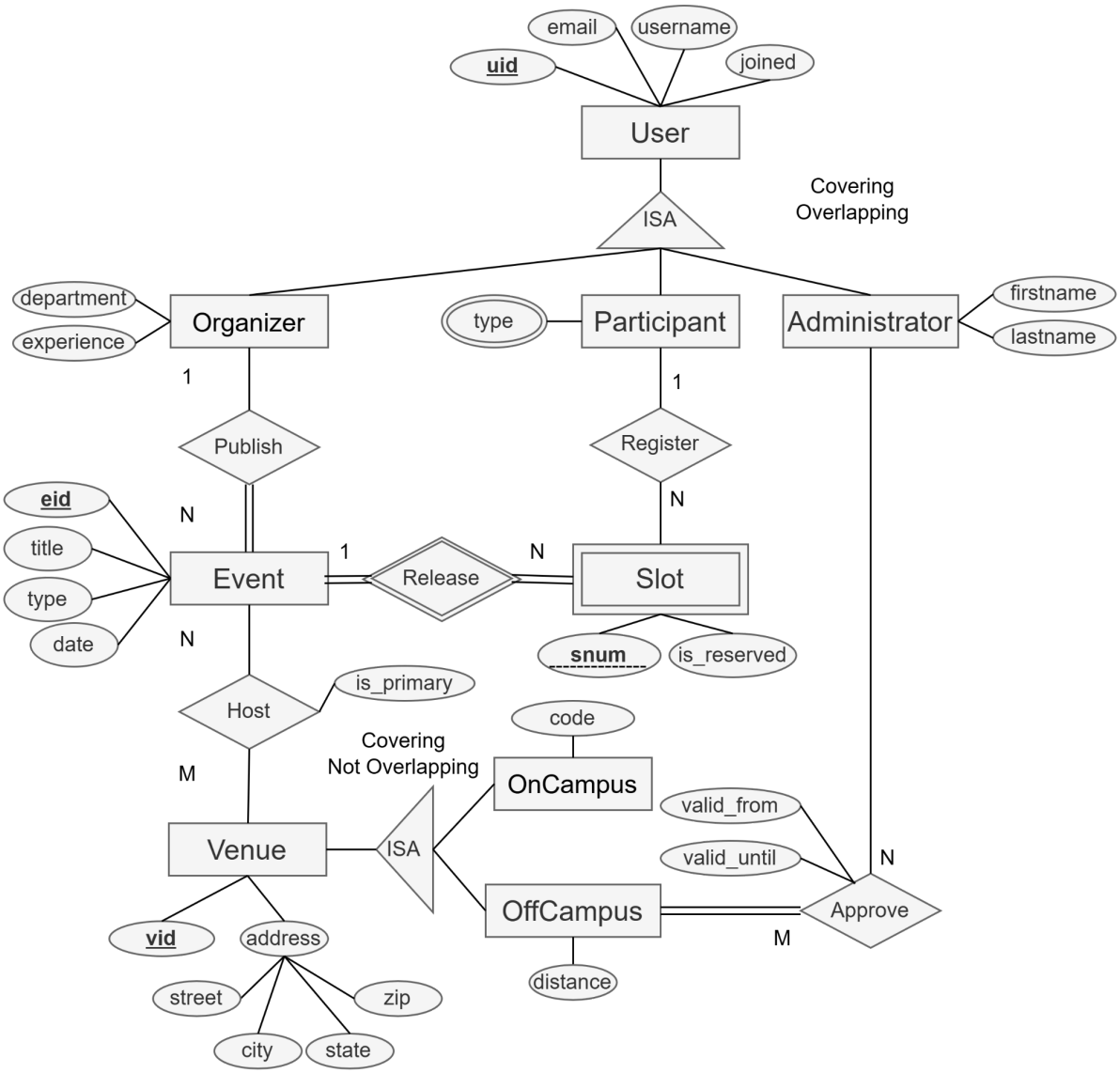
Hw2 Relational Translation

Objective

In this assignment, you will design a relational schema for **ZotEvent,** an online platform dedicated to event organising for anteaters. This schema will be based on the ER model designed in HW1. The database schema will ensure efficient organization of User, Organizer, Participant, Administrator, Event, Slot, Venue, and OnCampus/OffCampus Venues.

You are tasked with **creating SQL DDL statements for each table**, **capturing all entities and relationships described in the ER model**. The schema will support the platform's functionality, including user roles, video publishing, streaming sessions, and review mechanisms. The database must adhere to best practices, maintain referential integrity, and represent all relationships and constraints from the ER diagram.



Details:

User and Organizer / Participant / Administrator (40 points):

- A user is uniquely identified by their uid.
- Each user has a username, email address, and a joining date.
- A user can act as an Organizer, Participant, Administrator, or any combination.
    - Organizers have additional attributes: department and experience (in years).
    - Participants have their preferred event types (represented as a single string, with each type in the list separated by a comma).
    - The administrator has firstname and lastname.

Event and Slot (20 points)

- An event is uniquely identified by the eid and includes a title, type, and a datetime.
- It should also have an additional attribute that reflects who the organizer of the event is.
- Slots are weak entities, uniquely identified by a combination of eid and snum.
- A slot should also contain the is_reserved flag and an additional attribute reflecting the registered participant.

Venue and OnCampus / OffCampus (20 points)

- A venue is uniquely identified by vid.
- It includes the address (a composition of street, city, state, and zip).
- It is further categorized into On-Campus (with code) or Off-Campus (with distance).

Relations (20 points)

- One table storing the ‘hosting’ relation between the event and the venue. It should also have an is_primary flag indicating that the venue is the main venue.
- Another table storing the ‘approval’ relation between the administrator and the off-campus venue. It should also record the dates valid_from and valid_until.

Guidelines

When creating your DB schema, use the following guidelines:

- Minimize the number of relations whenever possible.
- Use the “Delta Table” approach for ISA relationships.
- Derived attributes must be calculated via SQL queries or other mechanisms we have not learned yet. For now, enter a derived attribute as a basic attribute.
- For domain names (data types), use the most appropriate types for each column.

Example candidate data types:

Category	Type	Remark
NUMBER	INTEGER	A number type for integer values.
	DECIMAL(x,y)	A number type for real values where **x** is the maximum number of digits and **y** is the number of digits to the right of the decimal point.
STRING	CHAR(n)	A fixed-length string type where **n** is the column length.
	VARCHAR(n)	A variable-length string type where **n** specifies the maximum column length.
DATETIME	DATE	A type used for values with a date part but no time part. The format is '0000-00-00'.
	TIME	A type used for values with a time part. The format is '00:00:00'.
	DATETIME	A type used for values with both a date part and a time part. The format is '0000-00-00 00:00:00'.
ENUM	STRING	A string type with the value chosen from a list of permitted values.

Note that we are not specifying the exact data type/size for each attribute within the ER model. Use your best judgment on the value to choose. You will be graded based on the selection of an appropriate type (and size), not on an exact value.

Use the template (attached to the end of this document) as a mandatory starting point for your submission. Use only the entity, relationship, and attribute names from the final ER diagram while naming your tables and columns (to make it clear how your design corresponds to the ER diagram).

Create a SQL DDL statement for each table necessary based on the ER diagram. Make sure to capture the information and constraints of the ER diagram as faithfully as possible. Be sure to include the following information in the DDL statements for each table:

- name, columns, and column types (and sizes if applicable)
- primary keys column(s)
- foreign key column(s) (and indicate which table it references)
- *not null* constraints and/or other referential integrity constraints

Note: Fold relationship tables whenever possible!

Submission & Grading
Submit a PDF file to Gradescope. Make sure to only include the relevant information per question as specified below. Any information not included on the designated pages will not be graded.

The template has one page for entities and their supporting tables and one page for relationships DDLs. Feel free to append more pages at the end of each part if you need more space.

- Question 1 page(s): List the SQL DDL statements for creating tables for entities, including any supporting tables for entity-related information.
    - Any table with a foreign key constraint should be declared AFTER the creation of the referenced table.
- Question 2 page(s): List additional SQL DDL statements to create any additional tables for relationships.

Mark the pages in Gradescope!
Make sure each question starts on a new page!
Make sure to test your DDL statements in MySQL!

Template

Please copy the content below to start!

```
DROP DATABASE IF EXISTS cs122a_hw2;
CREATE DATABASE cs122a_hw2;
USE cs122a_hw2;

-- Define entities and their supporting tables below here


-- Q1: User and Organizer/Participant/Administrator (SQL DDL for entity(ies) table(s) only)

-- User Table (10 points)
CREATE TABLE User (
    uid INT,
    email TEXT NOT NULL,
    username TEXT NOT NULL,
    joined DATE NOT NULL,
    PRIMARY KEY (uid)
);

-- Organizer (Delta Table for ISA Relationship) (10 points)
CREATE TABLE Organizer (
    uid INT,
    department TEXT NOT NULL,
    experience INT NOT NULL,
    PRIMARY KEY (uid),
    FOREIGN KEY (uid) REFERENCES User(uid) ON DELETE CASCADE
);

-- Participant (Delta Table for ISA Relationship) (10 points)
CREATE TABLE Participant (
    uid INT,
    type TEXT,
    PRIMARY KEY (uid),
    FOREIGN KEY (uid) REFERENCES User(uid) ON DELETE CASCADE
);

-- Administrator (Delta Table for ISA Relationship)(10 points)
CREATE TABLE Administrator (
    uid INT,
    firstname TEXT NOT NULL,
    lastname TEXT NOT NULL,
    PRIMARY KEY (uid),
    FOREIGN KEY (uid) REFERENCES User(uid) ON DELETE CASCADE
);
```

```
-- Q2: Event and Slot (SQL DDL for entity(ies) table(s) only)

-- Event Table(10 points)
CREATE TABLE Event (
    eid INT,
    creator_uid INT NOT NULL,
    title TEXT NOT NULL,
    type TEXT NOT NULL,
    datetime DATETIME NOT NULL,
    PRIMARY KEY (eid),
    FOREIGN KEY (creator_uid) REFERENCES Organizer(uid) ON DELETE CASCADE
);

-- Slot Table (Weak Entity)(10 points)
CREATE TABLE Slot (
    eid INT,
    snum INT NOT NULL,
    is_reserved BOOLEAN NOT NULL,
    uid INT,
    PRIMARY KEY (eid, snum),
    FOREIGN KEY (eid) REFERENCES Event(eid) ON DELETE CASCADE,
    FOREIGN KEY (uid) REFERENCES Participant(uid) ON DELETE CASCADE
);

-- Q3: Venue and OnCampus / OffCampus (SQL DDL for entity(ies) table(s) only)

-- Venue Table (10 points)
CREATE TABLE Venue (
    vid INT,
    street TEXT NOT NULL,
    city TEXT NOT NULL,
    state TEXT NOT NULL,
    zip TEXT NOT NULL,
    PRIMARY KEY (vid)
);

-- OnCampus Table (5 points)
CREATE TABLE OnCampus (
    vid INT,
    code TEXT NOT NULL,
    PRIMARY KEY (vid),
    FOREIGN KEY (vid) REFERENCES Venue(vid) ON DELETE CASCADE
);

-- OffCampus Table (5 points)
CREATE TABLE OffCampus (
    vid INT,
    distance INT NOT NULL,
    PRIMARY KEY (vid),
```

```
    FOREIGN KEY (vid) REFERENCES Venue(vid) ON DELETE CASCADE
);

-- Q4: Write additional SQL DDL for relationship(s) table(s) below (20 points)

-- Hosting Relation Table (10 points)
CREATE TABLE Hosting (
    eid INT NOT NULL,
    vid INT NOT NULL,
    is_primary BOOLEAN NOT NULL,
    PRIMARY KEY (eid, vid),
    FOREIGN KEY (eid) REFERENCES Event(eid) ON DELETE CASCADE,
    FOREIGN KEY (vid) REFERENCES Venue(vid) ON DELETE CASCADE
);

-- Approval Relation Table (10 points)
CREATE TABLE Approval (
    uid INT NOT NULL,
    vid INT NOT NULL,
    valid_from DATE NOT NULL,
    valid_until DATE NOT NULL,
    PRIMARY KEY (uid, vid),
    FOREIGN KEY (uid) REFERENCES Administrator(uid) ON DELETE CASCADE,
    FOREIGN KEY (vid) REFERENCES OffCampus(vid) ON DELETE CASCADE
);
```